



Containerization with Docker

Introduction

What is Docker?

Docker is a tool that promises to easily encapsulate the process of creating a distributable artifact for any application, deploying it at scale into any environment, and streamlining the workflow and responsiveness of agile software organizations.

Docker Image vs Docker Container

An *image* is a template of a container. One image can be used to build multiple containers. You can also create multiple versions of an image by adding tags.

A *container* is a lightweight virtual environment that is created using an image. It is highly portable and runs on top of the underlying OS kernel.

What can Docker be used for?

Containers are great for setting up automated CI-CD workflows. Docker makes it easy to configure consistent environments using images and containers for development, testing and production and is developer-friendly!

Consider the following example scenario:

Your developers write code locally and share their work with their colleagues using Docker containers. They use Docker to push their applications into a test environment and execute automated and manual tests. When developers find bugs, they can fix them in the development environment and redeploy them to the test environment for testing and validation. When testing is complete, getting the fix to the customer is as simple as pushing the updated image to the production environment.

What is a Docker Registry?

A docker registry such as DockerHub is a library or repository for storing, versioning and distributing docker images.

DockerHub (<https://hub.docker.com/>) is a public docker registry where you can find official container images and also upload your own custom images and set up private repositories. It can also be used to automatically build and push an image each time you push new code to your source provider as part of DevOps workflows.

Setting Up

Docker Desktop is a one-click-install application for your Mac, Linux, or Windows environment that enables you to build and share containerized applications and



microservices. It provides a straightforward GUI (Graphical User Interface) to manage your containers, applications, and images directly from your machine. Docker Desktop can be used either on its own or as a complementary tool to the CLI.

Installing Docker on Mac

Follow the steps here: (<https://docs.docker.com/desktop/install/mac-install/>) to install it on your Mac machine. Make sure also to check the required permissions and configurations here: (<https://docs.docker.com/desktop/mac/permission-requirements/>)

Installing Docker on Linux

Check the required permissions and configurations here: (<https://docs.docker.com/desktop/install/linux-install/>) to install it on your Linux machine.

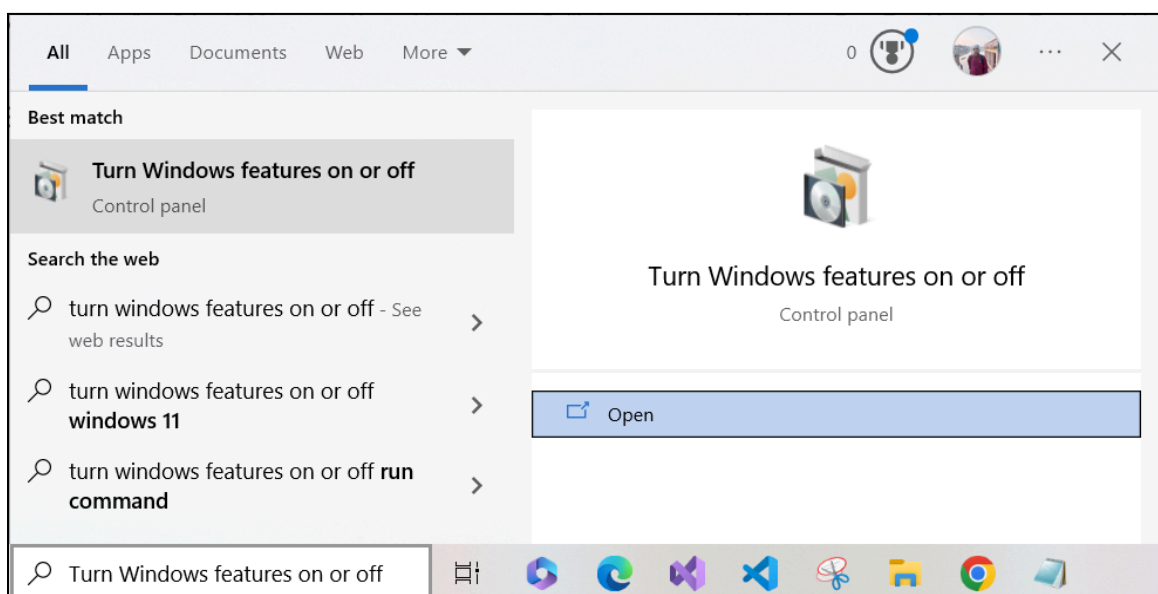
Installing Docker on Windows

Steps include:

1. Ensure you have Windows 10 version 1903 or higher, or Windows 11.
2. Using Windows Subsystem for Linux (WSL) 2 feature, install the Linux distribution.
3. Download and install Docker Desktop for Windows.

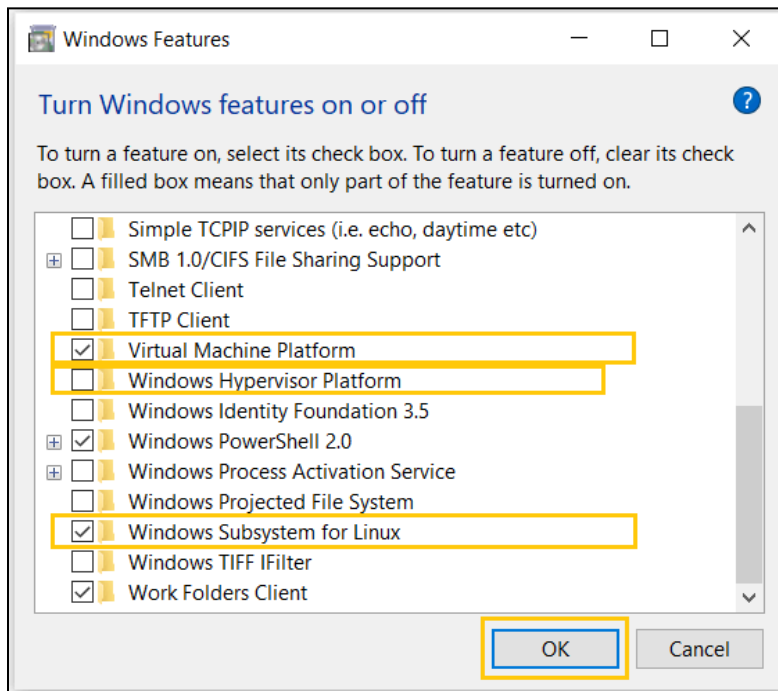
Enable Windows Subsystem for Linux (WSL) 2 feature on Windows

1. In the Windows Search bar, type “Turn Windows features on or off”, and open it.





2. Enable “Virtual Machine Platform”, and “Windows Subsystem for Linux”.
Make sure “Windows Hypervisor Platform” is disabled. Then click OK.



3. It will ask for a Restart. Restart your machine for changes to take effect.
4. Open PowerShell or Windows Command Prompt in administrator mode by selecting “Run as administrator”, enter the below command:

```
wsl --install -d Ubuntu-20.04
```

This command will enable the features necessary to run WSL and install the Ubuntu distribution of Linux.

You'll be asked to wait for files to decompress and be stored on your machine.

```
PS C:\WINDOWS\system32> wsl --install -d Ubuntu-20.04
Installing: Ubuntu 20.04 LTS
Ubuntu 20.04 LTS has been installed.
Launching Ubuntu 20.04 LTS...
Installing, this may take a few minutes...
```

You will need to create a user account and password for your newly installed Linux distribution.



```
PS C:\WINDOWS\system32> wsl --install -d Ubuntu-20.04
Installing: Ubuntu 20.04 LTS
Ubuntu 20.04 LTS has been installed.
Launching Ubuntu 20.04 LTS...
Installing, this may take a few minutes...
^@^@Please create a default UNIX user account. The username does not need to match your Windows username.
For more information visit: https://aka.ms/wslusers
Enter new UNIX username: testuser
New password:
Retype new password:
passwd: password updated successfully
Installation successful!
```

Once done, the terminal will start for your newly installed Linux distribution.

```
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

Welcome to Ubuntu 20.04.3 LTS (GNU/Linux 5.15.90.1-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Thu Mar 30 18:15:26 IST 2023

System load:  0.83               Processes:            9
Usage of /:   0.1% of 1006.85GB  Users logged in:     0
Memory usage: 4%                IPv4 address for eth0: 172.26.50.91
Swap usage:   0%

1 update can be applied immediately.
To see these additional updates run: apt list --upgradable

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

This message is shown once a day. To disable it please create the
/home/testuser/.hushlogin file.
testuser@LAPTOP-SAGARNCF8ITIA:~$
```

5. List the installed Linux distribution and check the version of WSL it is set to using the below command:

`wsl -l -v` (within PowerShell)

`wsl.exe -l -v` (within the terminal)

```
testuser@LAPTOP-SAGARNCF8ITIA:~$ pwd
/home/testuser
testuser@LAPTOP-SAGARNCF8ITIA:~$
testuser@LAPTOP-SAGARNCF8ITIA:~$ wsl.exe -l -v
  NAME          STATE          VERSION
* Ubuntu-20.04  Running        2
```



6. If WSL version is 1, set the default version to WSL 2 using the below command:

```
wsl --set-default-version 2
```

```
testuser@LAPTOP-SAGARNCF8ITIA:~$ wsl.exe --set-default-version 2
For information on key differences with WSL 2 please visit https://aka.ms/wsl2
The operation completed successfully.
```

Install Docker Desktop for Windows

1. Download Docker Desktop from the link below:

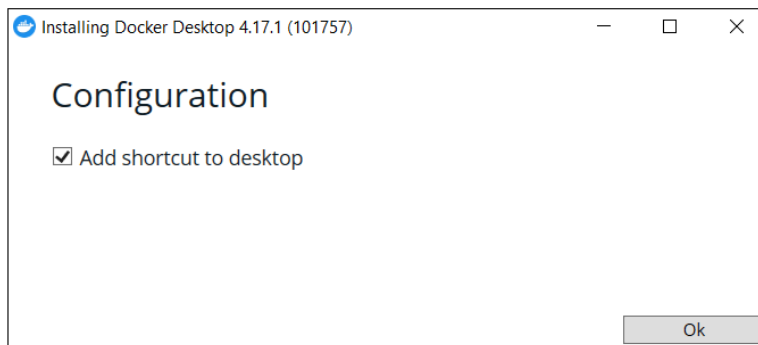
<https://www.docker.com/products/docker-desktop/>

For Windows using the link below will directly download the installer.

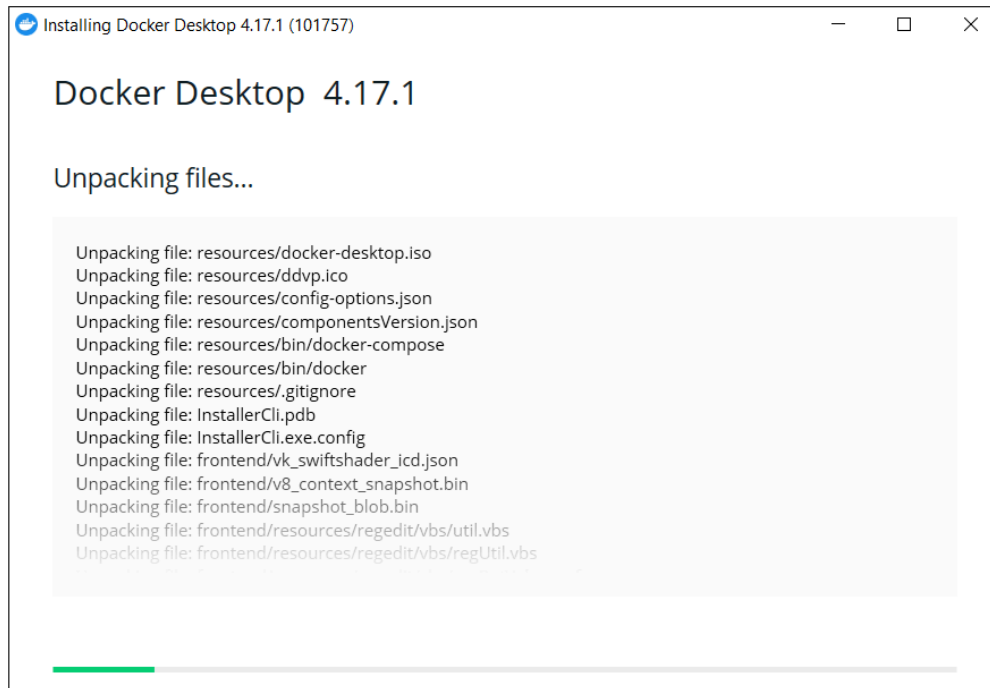
<https://desktop.docker.com/win/main/amd64/Docker%20Desktop%20Installer.exe>

2. Start the installer.

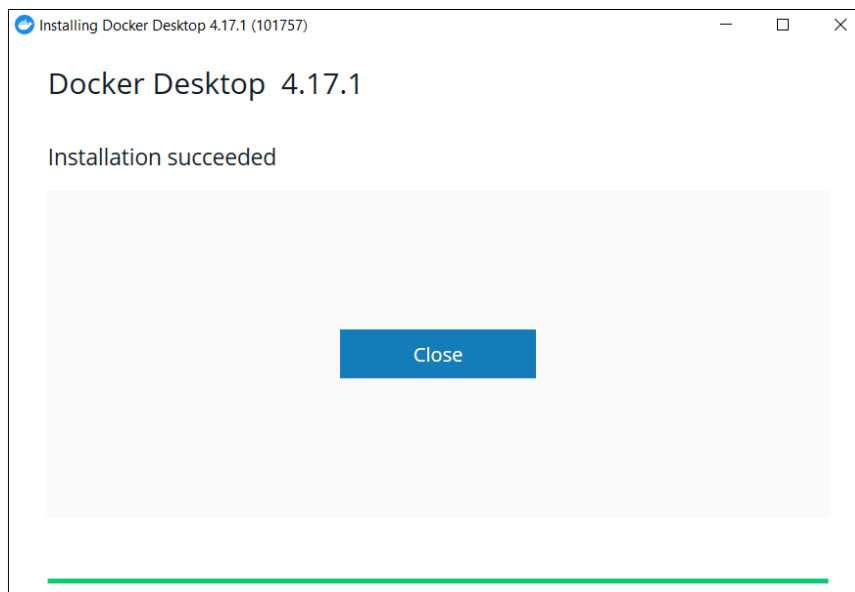
Add the shortcut to the desktop.



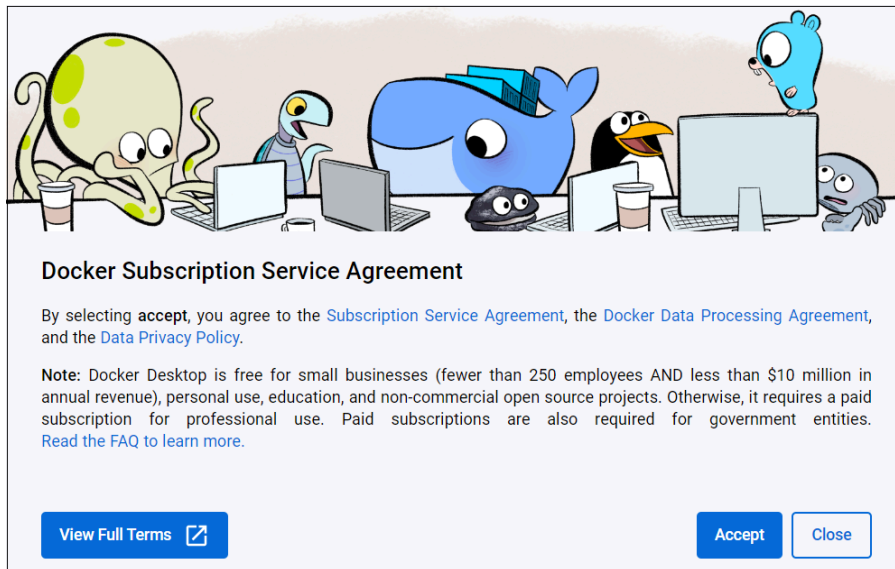
It will start unpacking files.



Once installation is complete, close the wizard.



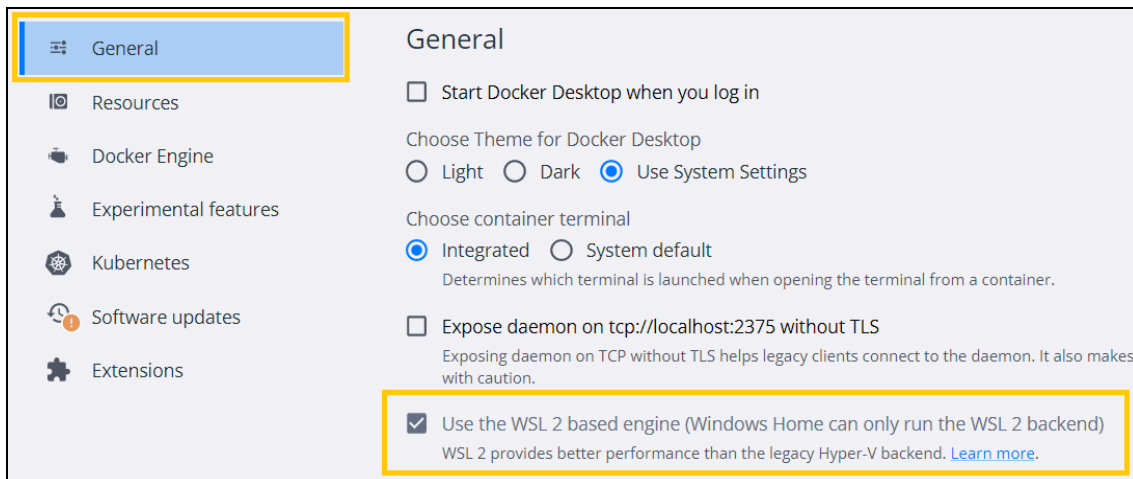
May ask for restarting the system.



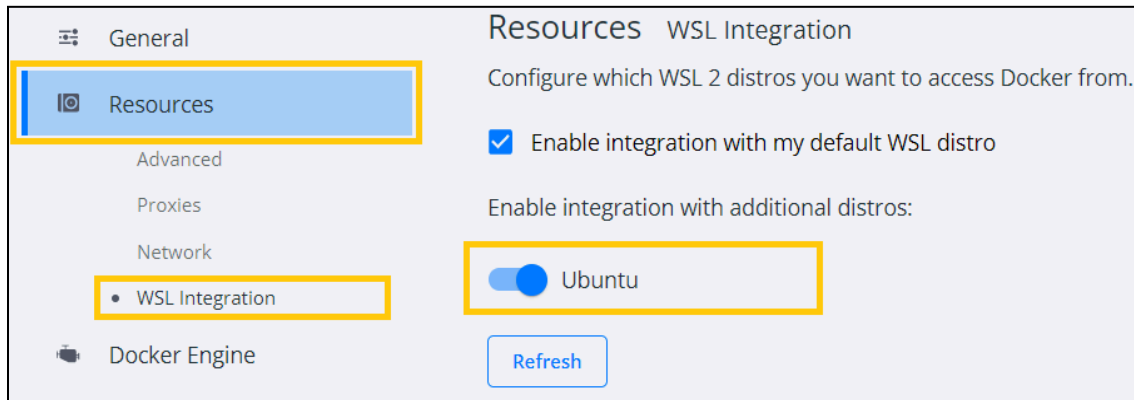
Accept the agreement. Skip the “Tell us about the work you do” if asked. Close it.

3. Start Docker Desktop using the shortcut created on the desktop.

Go to the → **Settings** [left of Sign in logo] → **General** → **Use the WSL-based engine** should be enabled or enable it.



Go to the → **Resources** → **WSL integration** → **Enable integration with additional distro**



To check whether it is installed and working correctly: open cmd and type → **wsl**

Now in linux terminal type → **docker run hello-world**

You will get message like this if installed correctly:

Hello from Docker!

This message shows that your installation appears to be working correctly.

```
yrajm1997@DESKTOP-DLM8PVA:/mnt/c/Users/hp$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
719385e32844: Pull complete
Digest: sha256:a13ec89cdf897b3e551bd9f89d499db6ff3a7f44c5b9eb8bca40da20eb4ea1fa
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.
```

Check the Docker Desktop, you should see the container which was running the image “hello-world”.

Same thing can be checked with window cmd also as given below:

docker run hello-world

```
C:\Users\user>docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
2db29710123e: Pull complete
Digest: sha256:ffb13da98453e0f04d33a6eee5bb8e46ee50d08ebe17735fc0779d0349e889e9
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.
```

Hence, Docker Desktop is installed successfully.



Installing Docker on Ubuntu

1. Open the terminal.
2. Remove any Docker files that are running in the system, using the following command:

```
sudo apt-get remove docker docker-engine docker.io
```

After entering the above command, you will need to enter the password of the root user and press enter.

3. Get the system up-to-date using the following command:

```
sudo apt-get update
```

4. Install Docker using the following command:

```
sudo apt install docker.io
```

You'll get a prompt asking you to choose between y/n - choose y

5. Start the docker service and enable the service to start at boot using the following commands:

```
sudo systemctl start docker
```

```
sudo systemctl enable docker
```

6. Before testing Docker, check the version installed using the following command:

```
docker --version
```

```
ubuntu@ip-172-31-82-246:~$ docker --version
Docker version 20.10.12, build 20.10.12-0ubuntu4
```

You can also check full version details for Docker Client and Server using:



sudo docker version

```
ubuntu@ip-172-31-81-241:~$ docker version
Client:
 Version:           20.10.12
 API version:       1.41
 Go version:        go1.17.3
 Git commit:        20.10.12-0ubuntu4
 Built:             Mon Mar  7 17:10:06 2022
 OS/Arch:           linux/amd64
 Context:           default
 Experimental:      true

Server:
 Engine:
  Version:          20.10.12
  API version:      1.41 (minimum version 1.12)
  Go version:       go1.17.3
  Git commit:       20.10.12-0ubuntu4
  Built:            Mon Mar  7 15:57:50 2022
  OS/Arch:          linux/amd64
  Experimental:     false
 containerd:
  Version:          1.5.9-0ubuntu3
  GitCommit:
 runc:
  Version:          1.1.0-0ubuntu1
  GitCommit:
 docker-init:
  Version:          0.19.0
  GitCommit:
```

7. Pull an image from the Docker hub using the following command:

sudo docker run hello-world

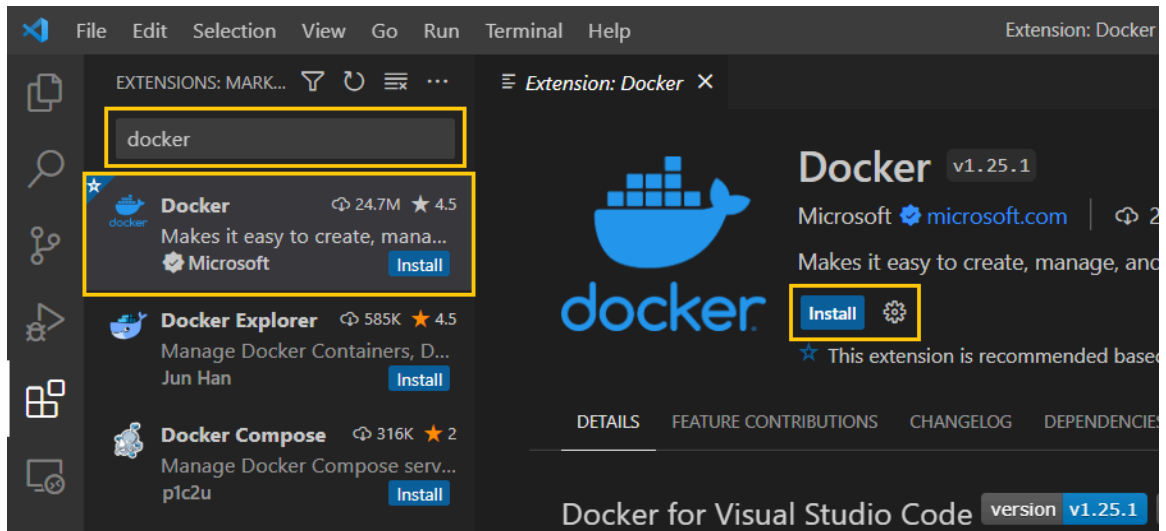
```
ubuntu@ip-172-31-82-246:~$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
2db29710123e: Pull complete
Digest: sha256:7d246653d0511db2a6b2e0436cfd0e52ac8c066000264b3ce63331ac66dca625
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.
```

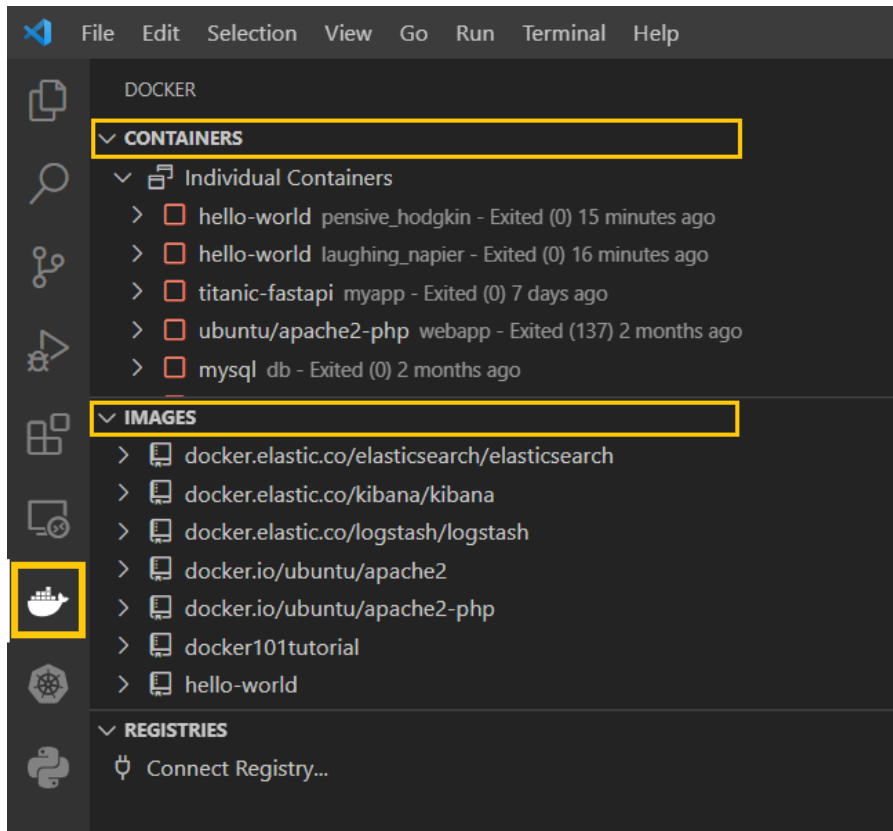
Here, hello-world is an existing docker image which should start running in a new container.

Setting Docker in VS-Code and running with Linux

1. Open VS-Code. Go to Extensions tab and install Docker extension.



2. Once installed, you will see the docker icon on the left panel. Click on that icon, and it will show the current containers, images which are present on your system.



3. To execute docker commands, go to *Terminal* > *New Terminal*. Type `wsl`, and start executing.

```
PS F:\Demo> wsl
yrajm1997@DESKTOP-DLM8PVA:/mnt/f/Demo$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
da78138fd248	hello-world	"/hello"	24 minutes ago	Exited (0) 24 minutes ago		pensive_hodgkin
94f8ff449971	hello-world	"/hello"	24 minutes ago	Exited (0) 24 minutes ago		laughing_napier
85ceedea80b3	titanic-fastapi	"python y"	7 days ago	Exited (0) 7 days ago		myapp
01ab00aabd16	ubuntu/apache2-php	"apache2d"	2 months ago	Exited (137) 2 months ago		webapp
848c051febd7	mysql	"docker-.s..."	2 months ago	Exited (0) 2 months ago		db
ac4339a33197	vault	"docker-.s..."	4 months ago	Exited (0) 2 months ago		vaultcont

Exploring the Basics

Docker Images and Tags

A docker image is identified by its image name which consists of two parts, `name:tag`



The name is the general name of the image and will ultimately be the image repository name when and if it is shared. An image name may contain lowercase letters, digits and separators, but it cannot contain underscores.

It is optionally prefixed by a registry hostname, `registry username/image name`

e.g. For the image name `appdynamics/sample-app-web`, `appdynamics` is the username of the registry host and `sample-app-web` is the name of the image and image repository name.

Docker tags convey useful information about a specific image version/variant. You can create multiple images (a group of images) which share the same name, but have more specialized properties. Hence, you can use a tag to define and identify a specialized version of an image. If a tag name is not specified while creating an image, the default value is applied i.e. `latest`.

A tag name may contain lowercase and uppercase letters, digits, underscores, periods and dashes. A tag name may not start with a period or a dash and may contain a maximum of 128 characters.

Hence an image name shared through a registry will usually look like this:

`username/image name:tag name`

e.g. `appdynamics/sample-app-web:latest`

e.g. `atlassian/confluence-server:latest`, `atlassian/confluence-server:eap-jdk11`, `atlassian/confluence-server:8.0.0-m90-ubuntu-jdk17` are versions of `confluence-server` image shared by `atlassian` on `DockerHub`.

e.g. `ubuntu:latest`, `ubuntu:jammy-20221101`, `ubuntu:22.10` are image names of `Ubuntu` versions available on `DockerHub` under the `ubuntu` repo.

Note: Docker commands need root privileges to be executed. Although you can login as the root user and run the commands directly, it is recommended to login as a non-root user for running these commands. In the following sections, we are logged in as 'ubuntu', hence the commands are prepended with '**sudo**' to grant root access for the `ubuntu` user to run these commands. If you do not use `sudo`, you might face access denied errors while running docker commands.

Pulling an Image from a Registry

Download a docker image by using the below command:

```
sudo docker pull <image name>:<tag>
```



e.g. `sudo docker pull appdynamics/sample-app-web`

Note: If the tag is not specified, the default tag i.e. latest will be used.

```
^Cubuntu@ip-172-31-82-246:~$ sudo docker pull appdynamics/sample-app-web
Using default tag: latest
latest: Pulling from appdynamics/sample-app-web
a3ed95caeb02: Pull complete
02b89e82a85f: Pull complete
73e034e1d5fb: Pull complete
b062d0988d0a: Pull complete
0e72317a6fba: Pull complete
e59a80a9af6e: Pull complete
1b7cf628accc: Pull complete
fb9d7acac92c: Pull complete
541be71d2dce: Pull complete
dcf4879a84cf: Pull complete
d6ce0478e82f: Pull complete
2554cd590bfa: Pull complete
99e6cfa2bfbe: Pull complete
de65d3764bf9: Pull complete
Digest: sha256:a0c63b33c9aef367e564b059fc6f4a4a249668ff5de52fd467ea534161856616
Status: Downloaded newer image for appdynamics/sample-app-web:latest
docker.io/appdynamics/sample-app-web:latest
```

Checking Available Images

`sudo docker images` or `sudo docker image ls`

```
ubuntu@ip-172-31-82-246:~$ sudo docker images
REPOSITORY          TAG         IMAGE ID      CREATED       SIZE
hello-world         latest     feb5d9fea6a5  11 months ago 13.3kB
appdynamics/sample-app-web latest     3904a3895bd2  5 years ago   627MB
```

Running a Container from an Image

`sudo docker run -it -p <port no.>:<container port no.> -d --name <new container name> <image name>`

e.g. `sudo docker run -p 3000:3000 -d --name sample-app appdynamics/sample-app-web`

If your container is hosting a web server or app like in the given example, you should be able to access it through your browser using `http://[Your Public/Localhost IP]:[Host port no.]`

Note:

- `-d` stands for detached mode which means the container will run in background and you can keep running commands on the terminal
- `-p` or `--publish` is used for publishing the specified container port to the host
- use `--name` option for giving a name to your container
- `-it` stands for interactive terminal mode
- each container will be assigned a unique container id

```
ubuntu@ip-172-31-82-246:~$ sudo docker run -p 3000:3000 -d --name sample-app appdynamics/sample-app-web
217a830f62cca12cd359970b53810990c86c5557aad96a17a3cfe4813a5e81e9
```



Listing Active Docker Containers

The below command will list all running containers i.e. active containers with status as 'up'.
Test if your container process is running and mark its container id and name.

`sudo docker ps`

```
ubuntu@ip-172-31-81-241:~$ docker ps
```

CONTAINER ID	IMAGE	NAMES	COMMAND	CREATED	STATUS	PORTS
f0d161800a35	appdynamics/sample-app-web	sample-app	"/bin/sh -c 'install..."	2 minutes ago	Up 2 minutes	0.0.0.0:3000->3000/tcp, :::3000->3000/tcp

To list all active containers, run the following command:

`sudo docker ps -a`

```
ubuntu@ip-172-31-81-241:~$ docker ps -a
```

CONTAINER ID	IMAGE	NAMES	COMMAND	CREATED	STATUS
f0d161800a35	appdynamics/sample-app-web	sample-app	"/bin/sh -c 'install..."	11 seconds ago	Up 9 seconds
9c36ad0f6bb0	linuxserver/wireshark	wireshark	"/init"	5 days ago	Exited (0) 35 seconds ago

Notice the values listed under the 'Container ID', 'Image', and 'Names' columns:

- The Container ID is a unique identifier given to a container.
- The Image column will display the name of the image used to run the container.
- The Name column displays a unique name for the container which can be assigned using the name flag. If no name is provided by the user while creating/running a Docker container, Docker automatically assigns the container a name.

Interacting with Docker Containers

You can open the bash terminal of your running container to interact with it using the below command:

`sudo docker exec -it <container id or name> bash`

Here you can try running shell commands, installing software etc within your container. To exit out of the shell session, simply type exit.

```
ubuntu@ip-172-31-82-246:~$ sudo docker exec -it 217a830f62cc bash
[root@217a830f62cc /]# ping 127.0.0.0
PING 127.0.0.0 (127.0.0.0) 56(84) bytes of data:
64 bytes from 127.0.0.0: icmp_seq=1 ttl=64 time=0.022 ms
64 bytes from 127.0.0.0: icmp_seq=2 ttl=64 time=0.036 ms
64 bytes from 127.0.0.0: icmp_seq=3 ttl=64 time=0.036 ms
^C
--- 127.0.0.0 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2257ms
rtt min/avg/max/mdev = 0.022/0.031/0.036/0.008 ms
[root@217a830f62cc /]# exit
exit
```

Committing Changes to a Docker Container



You can commit a container's file changes or settings into a new image by using the below command:

`sudo docker commit <container id or name> <new image name>`

```
ubuntu@ip-172-31-81-241:~$ sudo docker commit sample-app demo-image
sha256:71faf5a96bd6773ff2114565ae8aa29e82cf317517618ba73b0452237258a9e8
ubuntu@ip-172-31-81-241:~$ docker images
REPOSITORY          TAG         IMAGE ID      CREATED        SIZE
demo-image          latest     71faf5a96bd6  7 seconds ago  627MB
library/ubuntu     latest     30e033e0db0e  6 days ago    806MB
```

Stopping a Running Docker Container

You can stop a running container gracefully using the below command:

`sudo docker stop <container id or name>`

```
ubuntu@ip-172-31-81-241:~$ docker stop sample-app
sample-app
ubuntu@ip-172-31-81-241:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
ubuntu@ip-172-31-81-241:~$
```

Deleting Docker Containers

If you no longer want to use a container you can delete it using the below command. This will clear up the memory allocated for the container as well.

`sudo docker rm <container id or name>`

```
ubuntu@ip-172-31-81-241:~$ docker rm sample-app
sample-app
ubuntu@ip-172-31-81-241:~$
```

To delete a running container by force, you can use the -f switch.

`sudo docker rm -f <container id or name>`

```
ubuntu@ip-172-31-81-241:~$ sudo docker rm -f sample-app
sample-app
ubuntu@ip-172-31-81-241:~$
```

Deleting Docker Images

You can delete any unused images using the following command:

`sudo docker rmi <image name>`

```
ubuntu@ip-172-31-81-241:~$ sudo docker rmi demo-image
Untagged: demo-image:latest
Deleted: sha256:71faf5a96bd6773ff2114565ae8aa29e82cf317517618ba73b0452237258a9e8
Deleted: sha256:6ae4bbb9b06aab46516012541ce49bd756a83fc13267f69064af46ea1119c725
ubuntu@ip-172-31-81-241:~$
```

Using Docker Networks

You can allow multiple otherwise isolated containers to connect to each other by creating a docker network. By default, a bridge-type network is created. You can do so using the below command:

`sudo docker network create <network name>`



```
ubuntu@ip-172-31-90-220:~$ docker network create sample
d194a5bc1393387ef0887af0d7aca659a947536834799e25c68f6aaac13c9a48
```

To allow a container to use this network, you need to pass it as a flag in the docker run command while starting the container.

```
sudo docker run --network <network_name> <image_name>
```

```
ubuntu@ip-172-31-90-220:~$ docker run --name db -e MYSQL_ROOT_PASSWORD=root -itd --network sample mysql
5dd0fe024196ff99bedf87cea71399d1a4a4476376dec2394759a850c253fcfb
```

Dockerizing your app with Dockerfile

A Dockerfile is a text config file that is needed to dockerize an application. It will define how your container is built and hence should contain all the steps for setting up and configuring the environment and deploying your application within the container.

We will take an example of a simple Fastapi application to show how to dockerize your app with Dockerfile. This simple application will return the sum of two numbers given by the user.

Create a mywebapp folder and create a main.py file within it. Add the contents as shown below.

```
mkdir mywebapp
```

```
cd mywebapp
```

```
touch main.py
```

```
from fastapi import FastAPI
```

```
from fastapi.responses import JSONResponse
```

```
app = FastAPI()
```

```
@app.get("/add/{num1}/{num2}")
```

```
def index(num1: int, num2: int):
```

```
    return JSONResponse({'Sum': num1 + num2})
```

```
if __name__ == "__main__":
```

```
    import uvicorn
```

```
    uvicorn.run(app, host="0.0.0.0", port=8001)
```

For example:

```
from fastapi import FastAPI
from fastapi.responses import JSONResponse

app = FastAPI()

@app.get("/add/{num1}/{num2}")
def index(num1: int, num2: int):
    return JSONResponse({'Sum': num1 + num2})

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8001)
```

Your webapp project is now ready. Now follow the steps below to dockerize the sample website project, i.e. create a dockerfile which should have the steps to spin up a docker container with fastapi dependencies installed, running on Ubuntu OS, and which will also deploy the webapp.

1. Create a file named Dockerfile at the root of your application (within mywebapp folder)

```
touch Dockerfile
```

2. Update the Dockerfile with the following commands:

```
FROM python:3.10
```

```
ADD . /webapp/
```

```
RUN pip install --upgrade pip
```

```
RUN pip install fastapi uvicorn
```

```
EXPOSE 8001
```

```
CMD ["python", "webapp/main.py"]
```

For example:

```
# pull python base image
```

```
FROM python:3.10
```

```
# copy application files
```

```
ADD . /webapp/
```

```
# update pip
```

```
RUN pip install --upgrade pip
```

```
# install dependencies
```



```
RUN pip install fastapi uvicorn
# expose port for application
EXPOSE 8001
# start fastapi application
CMD ["python", "webapp/main.py"]
```

Note:

- **FROM** $\langle \text{image} \rangle$: $\langle \text{tag} \rangle$ command is used to specify the base image on which your container will be built. It must be the first non-comment instruction in the Dockerfile.
- **ADD** $\langle \text{source} \rangle$ $\langle \text{destination} \rangle$ instruction is used to copy files either from the local filesystem or from a remote URL. Here the ' .' signifies that we want to copy all the files within the folder into the specified container target folder '/webapp/'
- **RUN** $\langle \text{command} \rangle$ or **RUN** $["\langle \text{executable} \rangle", "\langle \text{param1} \rangle", "\langle \text{param2} \rangle"]$ is used to specify any shell commands to be executed
- You can use multiple commands in a single RUN statement using the syntax:
RUN $\langle \text{command1} \rangle$ & & \
 $\langle \text{command2} \rangle$ & & \
 $\langle \text{command3} \rangle$
- You can also execute multiple commands within your container using an executable script file:
ADD $\langle \text{file.sh} \rangle$ /tmp/ $\langle \text{file.sh} \rangle$
RUN /tmp/ $\langle \text{file.sh} \rangle$
- **EXPOSE** instruction informs Docker that the container listens on the specified network ports at runtime
- **CMD** or **ENTRYPOINT** instruction is used to specify the command to launch the process that you want to run within the container on instantiation.
- **ENV** allows to set the key value pair as shell/environment variables

Building an Image using Dockerfile

Following is the syntax for creating a new image from a Dockerfile:

```
sudo docker build <directory> -t <image-name>:<tag>
```

Run the below command to create a custom image for your webapp using the Dockerfile:

```
sudo docker build . -t mywebapp
```



```
yrajm1997@DESKTOP-DLM8PVA:/mnt/f/Demo/mywebapp$ ls
Dockerfile  main.py
yrajm1997@DESKTOP-DLM8PVA:/mnt/f/Demo/mywebapp$ docker build . -t mywebapp
[+] Building 49.7s (9/9) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 348B
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load metadata for docker.io/library/python:3.10
=> [internal] load build context
=> => transferring context: 686B
=> CACHED [1/4] FROM docker.io/library/python:3.10@sha256:a8462db480ec3a74499a297b1f8e07494
=> [2/4] ADD . /webapp/
=> [3/4] RUN pip install --upgrade pip
=> [4/4] RUN pip install fastapi uvicorn
=> exporting to image
=> => exporting layers
=> => writing image sha256:560651cc31eb51ef682d6b727c2efe2f48c3d117e659c7fbb1e7d5e2aa97bc00
=> => naming to docker.io/library/mywebapp
```

Note:

- The directory value should point to the folder where the Dockerfile is added which is usually the root folder of your application. Use '.' if you are running the command in the same directory as your Dockerfile.
- -t flag stands for tag and you can use it to provide the image name (optionally with a tag) for your custom image. The default tag is 'latest'.
- Every command or step in the Dockerfile is added as a new layer while building the container. This is achieved by creating and removing intermediate containers. Each layer is cached separately so that subsequent builds are much faster.

Once your custom image is ready, you can view it by listing the available docker images.

```
yrajm1997@DESKTOP-DLM8PVA:/mnt/f/Demo/mywebapp$ docker images
REPOSITORY          TAG         IMAGE ID      CREATED        SIZE
mywebapp             latest      560651cc31eb  4 minutes ago  1.03GB
```

You can also run a container from the image, and test the webapp (use the command given in the section - Running a Container from an Image)

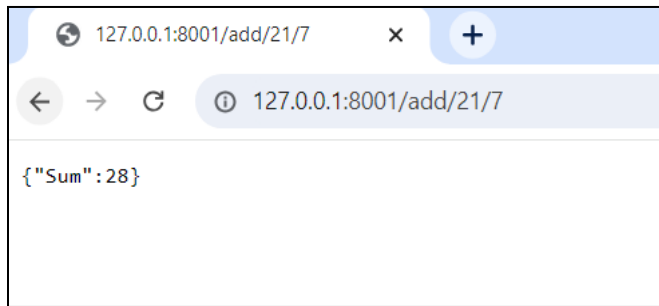
e.g. `docker run -p 8001:8001 -d --name=sample-app mywebapp`

```
yrajm1997@DESKTOP-DLM8PVA:/mnt/f/Demo/mywebapp$ docker run -p 8001:8001 -d --name=sample-app mywebapp
586b6e06fb717d4e9cff766c22918d4b534803cebd8a515041027e3be328208
yrajm1997@DESKTOP-DLM8PVA:/mnt/f/Demo/mywebapp$
yrajm1997@DESKTOP-DLM8PVA:/mnt/f/Demo/mywebapp$ docker logs -f sample-app
INFO:      Started server process [1]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8001 (Press CTRL+C to quit)
```

From the container logs, we can see that the application is running. To check your



application, visit <https://127.0.0.0:8001/add/21/7>.

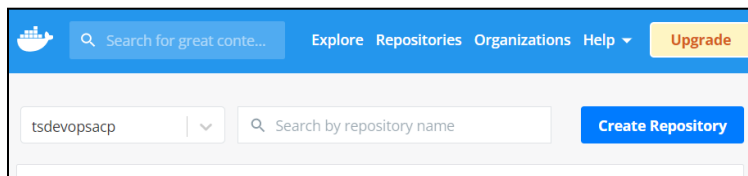


Sharing Images using an Image Library/Repository

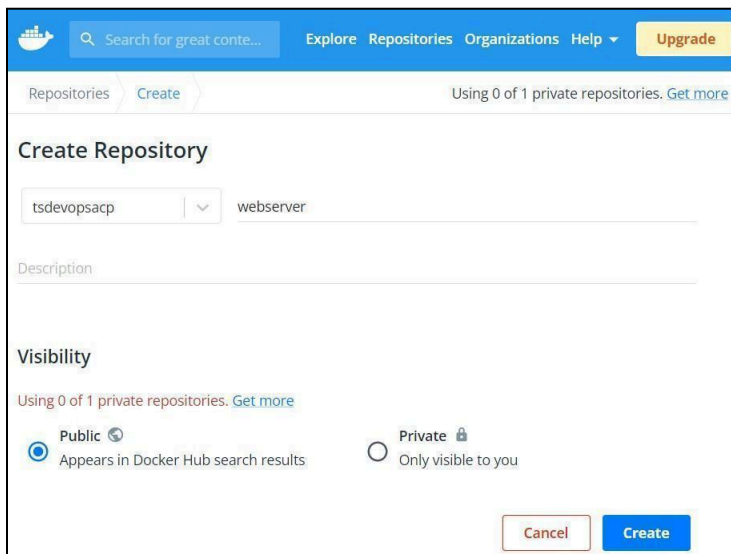
Sign up on DockerHub (<https://hub.docker.com/>) to start creating your repositories and storing custom images.

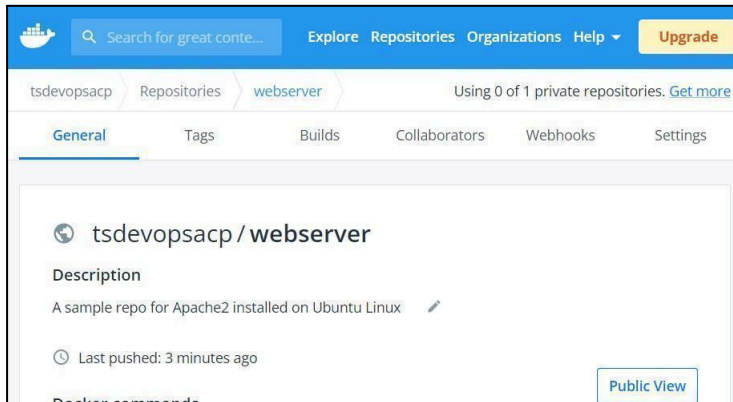
Creating a new Repository on DockerHub

Click on the 'Create Repository' button on your DockerHub Repositories tab.



Give a name for your repository. The name should match the name of your custom image.





Connecting to DockerHub through Terminal

Use the below command and your DockerHub credentials to login to DockerHub within your instance terminal.

`sudo docker login`

```
ubuntu@ip-172-31-82-246:/projects/mywebapp$ sudo docker login
Login with your Docker ID to push and pull images from Docker Hub. If you don't have a Docker ID,
head over to https://hub.docker.com to create one.
Username: tsdevopsacp
Password:
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
```

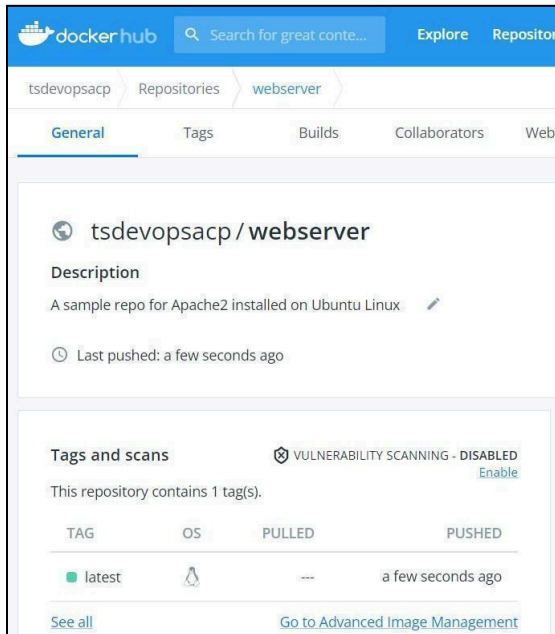
Pushing a Custom Image to DockerHub

Use the below command to push an image to your DockerHub repo. **Note:** The specified image name should match the full name of your DockerHub repository (username/repo).

`sudo docker push <image name>:<tag>`

```
ubuntu@ip-172-31-82-246:/$ sudo docker push tsdevopsacp/webserver:latest
The push refers to repository [docker.io/tsdevopsacp/webserver]
95622709caaa: Pushed
eb6df98b65ac: Pushed
389f01ac42aa: Pushed
68706b56e157: Pushed
629d9dbab5ed: Mounted from library/ubuntu
latest: digest: sha256:de47f28a55077e27dd78e9065bbea536b83938b1327f7a62f09e34485421f607 size: 1367
```

Your DockerHub repository should reflect the uploaded image and tag.



References

- Docker Docs <https://docs.docker.com/> - by Docker Inc.